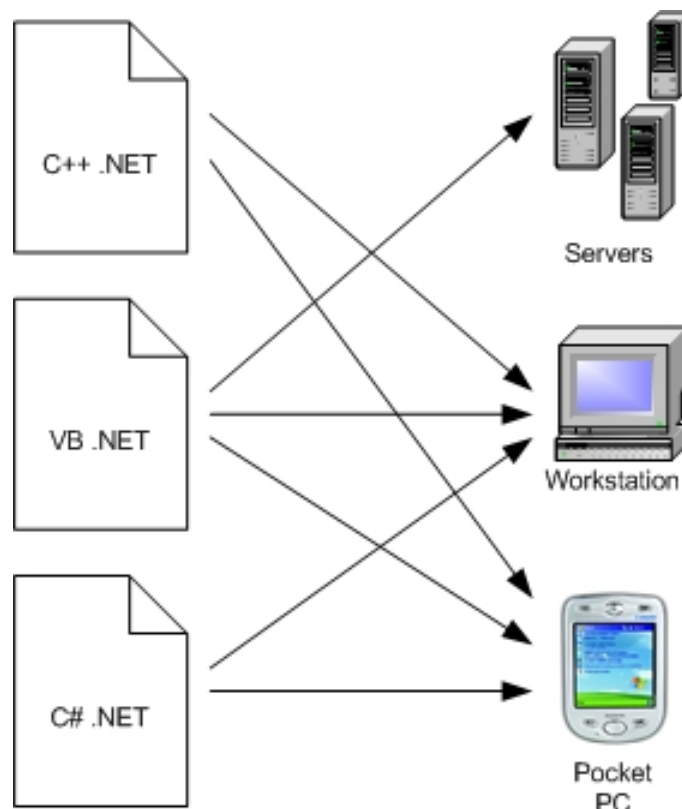


บทที่ 4

เทคโนโลยีของ .NET Framework

การพัฒนาโปรแกรมโดยใช้ .NET Framework นั้น เราสามารถเขียนโปรแกรมกับภาษาที่สนับสนุน .NET ที่เรามีความถนัด แล้วนำไปใช้งานกับระบบคอมพิวเตอร์ใดๆ ก็ได้ที่สนับสนุน .NET Framework ซึ่งมีคนมองว่าแนวคิดนี้คล้ายกับภาษา JAVA ของบริษัทซัน ไมโครซิสเต็มส์ จำกัด แต่ความจริงแล้วไม่ใช่เพราะภาษา JAVA นั้นเน้นการเขียนโปรแกรมครั้งเดียวด้วยภาษา JAVA แล้วสามารถทำงานได้ทุกแพลตฟอร์ม (ที่รองรับ JAVA) แต่ .NET Framework นั้นเราสามารถที่จะเลือกเขียนด้วยภาษาอะไรก็ได้ เพื่อให้ทำงานกับคอมพิวเตอร์ที่รองรับ .NET Framework (ในตอนนี้ยังมีแค่คอมพิวเตอร์ที่ใช้ระบบปฏิบัติการตระกูล Windows และใน Pocket PC เท่านั้น)



รูปที่ 4.1 การเขียนโปรแกรมกับ .NET Framework

4.1 ส่วนประกอบของ .NET Framework

.NET Framework นั้นประกอบไปด้วยส่วนประกอบหลักๆ 3 ส่วนคือ

4.1.1 Programming Language

เป็นภาษาที่ใช้ในการสร้างโปรแกรมซึ่งสามารถทำงานได้ภายใต้สถานะของ .NET โดยไมโครซอฟท์ได้เปิดตัวภาษาหลักๆ ที่จะใช้พัฒนามบน .NET เป็นจำนวน 4 ภาษาด้วยกันคือ

- C# เป็นภาษาใหม่ที่ไม่โครซอฟท์พัฒนามาจากภาษา C++ กับ JAVA เป็นหลัก
- VB .NET เป็นภาษาที่พัฒนามาจาก Microsoft Visual Basic 6.0
- C++ .NET เป็นภาษาที่พัฒนามาจาก Microsoft Visual C++ 6.0
- Jscript .NET เป็นภาษาที่พัฒนามาจาก Jscript ซึ่งเป็น JavaScript ในเวอร์ชันของไมโครซอฟท์

4.1.2 .NET Framework Classes Library

ไลบรารีนั้นเปรียบเสมือนชุดคำสั่งสำเร็จรูปย่อยๆ ที่ถูกจัดเตรียมไว้ โดยส่วนใหญ่จะเป็นชุดคำสั่งที่ใช้อยู่เป็นประจำ ซึ่งแต่ละไลบรารีภายในระบบ .NET Framework จะถูกจัดเก็บอยู่ในรูปของคลาสต่างๆ หรือที่เรียกว่าคลาสไลบรารีนั่นเอง

ซึ่งคลาสไลบรารี ของ .NET Framework นั้นเป็นส่วนที่รวบรวม Reusable Type ที่ถูกผสมผสานเข้ากับ Common Language Runtime โดยคลาสไลบรารีจะเป็นแบบ Object Oriented ซึ่งนอกจากจะทำให้เรียกใช้งานได้ง่ายแล้ว ยังจะช่วยลดทอนเวลาในการเรียนรู้ฟีเจอร์ใหม่ๆ ของ .NET Framework อีกด้วย นอกจากนี้แม้ในส่วนประกอบอื่นๆ (Third-Party Components) ก็ยังสามารถผสมผสานกับคลาสใน .NET Framework ได้อย่างกลมกลืนอีกด้วย

ยกตัวอย่างเช่น มีการจัดเตรียมกลุ่มคลาสของ .NET Framework เป็นกลุ่มหน้าจอดิตต่อกับผู้ใช้ (Interface) ที่สามารถนำมาใช้เพื่อพัฒนาเป็นกลุ่มคลาสในการเขียนโปรแกรมได้ โดยกลุ่มคลาสนี้จะรวมเข้ากับคลาสใน .NET Framework ได้อย่างกลมกลืน

โดยคลาสไลบรารีนั้นสนับสนุนในการพัฒนาโปรแกรมประยุกต์ต่างๆ ดังตัวอย่างต่อไปนี้

- Console Applications
- Windows GUI Applications (Windows Forms)
- ASP.NET Applications
- XML Web Services
- Windows Services

4.1.3 Common Language Runtime (CLR)

CLR นั้นถือได้ว่าเป็นองค์ประกอบที่สำคัญที่สุดของ .NET Framework เลยก็ว่าได้ เพราะว่า CLR นั้นมีหน้าที่ในการทำให้โปรแกรมที่เขียนขึ้นมาด้วยภาษาต่างๆ กลายเป็นภาษารูปแบบมาตรฐานเดียวกันหมด เราเรียกภาษาดังกล่าวว่า Microsoft Intermediate Language (MSIL หรือ IL) ซึ่งเมื่อเรานำโปรแกรมไปรันบนเครื่องใดๆ ที่ได้ติดตั้ง .NET Framework ลงไปแล้วนั้น CLR จะทำหน้าที่แปลง IL เป็นคำสั่งที่เหมาะสมต่อการทำงานของเครื่องนั้นๆ โดยทำการแปลงโปรแกรมเป็นภาษาเครื่องของเครื่องนั้นๆ นั่นเอง

โดย CLR นั้นจะทำหน้าที่จัดการกับหน่วยความจำ, การทำงานของเธรด, การทำงานของโค้ดที่เขียน, การตรวจสอบความปลอดภัยของโค้ด, การคอมไพล์ และบริการในระบบ ต่างๆ มากมาย ลักษณะการทำงานเหล่านี้เป็นส่วนหลักในการจัดการกับโค้ดบน Common Language Runtime

ด้วยการให้ความสำคัญกับความปลอดภัย ซึ่งขึ้นอยู่กับจำนวนของแพลตฟอร์มที่ประกอบกับจุดกำเนิดของตัวมันเอง (เช่น อินเทอร์เน็ต หรือเครือข่ายขององค์กร) นั้นหมายความว่าส่วนประกอบเพื่อการจัดการ (Managed Component) นั้นอาจจะสามารถทำการปฏิบัติการเข้าถึงไฟล์, รีจิสตรี หรือฟังก์ชันอื่นๆ หรือไม่ก็ได้ แม้ว่าอาจจะกำลังถูกใช้งานในแอปพลิเคชันที่ทำงานแบบเดียวกันอยู่ก็ตาม

สำหรับส่วนของ Runtime นั้นจะทำการจำกัดความปลอดภัยในการเข้าถึงโค้ด ยกตัวอย่างเช่น ผู้ใช้งานจะสามารถวางใจได้ว่าส่วน Embedded ในเว็บเพจจะสามารถแสดงภาพเคลื่อนไหวบนหน้าจอได้ แต่จะไม่สามารถเข้าถึงข้อมูลส่วนบุคคล, ระบบไฟล์ หรือส่วนเครือข่ายได้

อีกทั้งส่วน Runtime จะช่วยเพิ่มความปลอดภัยของโค้ดโดยการเตรียมโครงสร้างพื้นฐานในการตรวจสอบประเภทและโค้ด (Type-and-Code-Verification) ที่เรียกว่า Common Type System (CTS) การมี CTS ทำให้แน่ใจได้ว่าโค้ดเพื่อการจัดการทั้งหมดนั้นเป็น Self-Describing โดยคอมไพเลอร์ต่างๆ ของทั้งไมโครซอฟท์และในส่วนภาษาอื่น (Third-Party Language) จะทำการสร้างโค้ดเพื่อการจัดการที่คล้ายกันกับ CTS นั้นหมายความว่าโค้ดเพื่อการจัดการจะสามารถใช้ Instance และ Type เพื่อการจัดการอื่นๆ ได้อย่างเต็มที่ ขณะเดียวกันก็ช่วยเพิ่มความถูกต้องและความปลอดภัยของ Type อีกด้วย

นอกจากนี้ ในสภาวะแวดล้อมของการจัดการ (Managed Environment) ของส่วน Runtime จะทำการจัดปัญหาของซอฟต์แวร์ทั่วไปให้อีกด้วย ยกตัวอย่างเช่น ส่วน Runtime จะเข้าไปจัดการกับ Object Layout และ Manages References โดยอัตโนมัติ และจะปล่อยออกมาเมื่อพวกมันไม่ได้ถูกเรียกใช้งานอีกต่อไป ด้วยการเข้าไปจัดการหน่วยความจำแบบอัตโนมัตินี้จะช่วยแก้ปัญหา 2 ข้อ คือ การขาดแคลนหน่วยความจำ และการอ้างถึงหน่วยความจำผิดพลาด ซึ่งมักจะเกิดขึ้นในแอปพลิเคชันอยู่เป็นประจำ

4.2 ข้อดีของการใช้ .Net Framework ในการพัฒนาโปรแกรม

การใช้ .Net Framework ในการพัฒนาโปรแกรมนั้น มีข้อดีต่างๆ มากมายดังนี้

1) มีระบบไลบรารีที่เป็นมาตรฐานเดียวกัน

การใช้ .Net Framework มีระบบไลบรารีที่เป็นมาตรฐานเดียวกัน ทำให้เราไม่ต้องคอยกังวลว่าภาษาที่เราใช้เขียนนั้นมีไลบรารีตัวนั้นหรือตัวนี้หรือไม่ รวมถึงไม่ต้องคอยระวังว่าจะใช้ไลบรารีของภาษาหนึ่งแล้วอีกภาษาหนึ่งจะไม่มีไลบรารีตัวนี้

2) ไม่ขึ้นกับระบบปฏิบัติการ

เมื่อเครื่องคอมพิวเตอร์ที่เราใช้ในการพัฒนาได้ทำการติดตั้ง .NET Framework ลงไปแล้ว โปรแกรมที่เราพัฒนาโดยใช้ .NET Framework ก็จะสามารถทำงานบนเครื่องนั้นได้ทันที โดยไม่สนใจระบบปฏิบัติการบนเครื่องนั้นๆ

3) ใช้ภาษาในการพัฒนาได้ทุกภาษา

ทำให้เราไม่ต้องคอยมาศึกษาภาษาใหม่ๆ เมื่อต้องการสร้างโปรแกรมในแต่ละครั้ง นอกจากนี้เรายังสามารถเลือกใช้ภาษาที่เราถนัดที่สุดในการพัฒนาโปรแกรมต่างๆ ได้ด้วย ช่วยให้การพัฒนาโปรแกรมมีความสะดวกและง่ายดายขึ้นมาก

4) มีการควบคุมสถานะแวดล้อมในการทำงาน

ใน .Net Framework มีการควบคุมสถานะแวดล้อมในการทำงานเป็นอย่างดี เนื่องจากเป็นระบบที่เป็นมาตรฐาน ทำให้การควบคุมจัดสรรระบบต่างๆ ทำได้ง่าย เช่น การจัดสรรหน่วยความจำ การใช้งานเครื่องจะสามารถกระทำได้อย่างรวดเร็วขึ้น ลดโอกาสที่เครื่องจะทำงานผิดพลาดได้

5) มีความปลอดภัยสูง

ในการพัฒนาโปรแกรมบน .Net Framework นั้น เราสามารถที่จะกำหนดสิทธิในการใช้งานให้กับผู้ใช้งานได้ ซึ่งสามารถกำหนดได้ว่าโปรแกรมส่วนใดใช้งานได้หรือไม่ได้แล้วแต่เฉพาะบุคคลไป รวมทั้งมีฟังก์ชันในการเข้ารหัสและถอดรหัสข้อมูลอีกด้วย

4.3 การใช้ .NET Framework พัฒนาแอปพลิเคชันทางฝั่งเครื่องลูกข่าย

แอปพลิเคชันทางฝั่งลูกข่ายมีความใกล้เคียงเป็นอย่างมากกับแอปพลิเคชันแบบดั้งเดิมในการเขียนโปรแกรมโดยบน Windows-Based โดยแอปพลิเคชันนั้นจะมีรูปแบบการแสดงเป็นหน้าต่างหรือเป็นฟอร์มขึ้นมาบนเดสก์ท็อป เพื่อให้ผู้ใช้ใช้งาน แอปพลิเคชันทางฝั่งลูกข่ายนั้นจะรวมไปถึงโปรแกรมเวิร์ดหรือโปรแกรม Spreadsheet ต่างๆด้วย ซึ่งมักจะประกอบไปด้วยหน้าต่างสำหรับทำงาน, เมนู, ปุ่มกด หรือ ส่วนประกอบของ GUI ต่างๆ ซึ่งจะมีการเข้าถึงทรัพยากรภายในต่างๆ เช่น ระบบไฟล์ และอุปกรณ์ภายนอก เช่น เครื่องพิมพ์ เป็นต้น

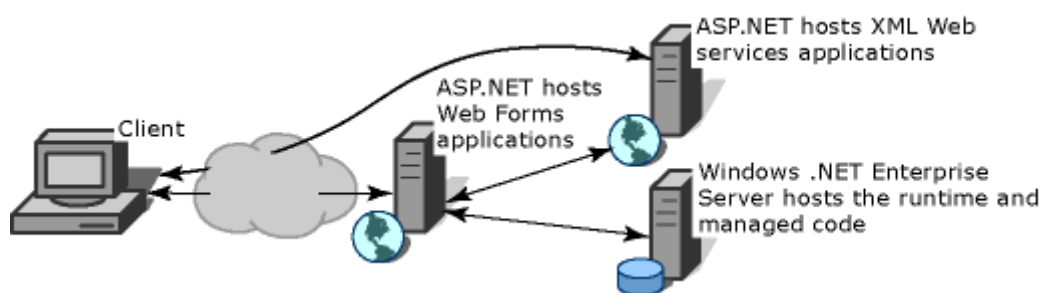
แอปพลิเคชันทางฝั่งลูกข่ายอีกแบบหนึ่งจะเป็น ActiveX control แบบดั้งเดิม (ในปัจจุบันถูกแทนที่ด้วย Windows Forms Control) ซึ่งใช้งานในอินเทอร์เน็ต ดังเช่น เว็บเพจต่างๆ ซึ่งแอปพลิเคชันแบบนี้จะมีส่วนคล้ายกับแบบอื่นๆ คือ ทำงานแบบเป็น Natively, มีการเข้าถึงทรัพยากรภายใน และประกอบไปด้วยส่วนประกอบที่เป็นภาพกราฟิก

ที่ผ่านมานั้นผู้พัฒนาโปรแกรมจะทำการสร้างแอปพลิเคชันโดยใช้ภาษา C หรือ C++ รวมเข้ากับ Microsoft Foundation Classes (MFC) หรือกับสถานะแวดล้อมแบบ Rapid Application Development (RAD) ดังเช่น Microsoft Visual Basic ซึ่งใน .NET Framework ได้ทำการรวมลักษณะของผลิตภัณฑ์ต่างๆ นั้นไว้เป็นหนึ่งเดียว ซึ่งมีสถานะแวดล้อมในการพัฒนาโปรแกรมที่สอดคล้องกันทั้งหมด ทำให้การพัฒนาแอปพลิเคชันทางฝั่งลูกข่ายทำได้ง่ายเป็นอย่างยิ่ง

4.4 การใช้ .NET Framework พัฒนาแอปพลิเคชันทางฝั่งเครื่องเซิร์ฟเวอร์

แอปพลิเคชันทางฝั่งเครื่องเซิร์ฟเวอร์นั้นจะถูกพัฒนาผ่าน Runtime Hosts ซึ่งจะทำการอนุญาตให้โค้ดเพื่อการจัดการที่กำหนดขึ้นทำการควบคุมพฤติกรรมของเซิร์ฟเวอร์ ในโมเดลนี้จะใช้งานพีเจิร์ชของ Common Language Runtime และคลาสไลบรารี เพื่อเพิ่มประสิทธิภาพและความเสถียรของเซิร์ฟเวอร์แม่ข่าย

รูปที่ 4.2 ด้านล่างนั้นแสดงถึงแบบเครือข่ายพื้นฐานซึ่งมีโค้ดเพื่อการจัดการกำลังทำงานในสถานะแวดล้อมของเซิร์ฟเวอร์ที่แตกต่างกัน ในเซิร์ฟเวอร์อย่างเช่น IIS และ SQL Server จะสามารถทำปฏิบัติการพื้นฐานได้ในขณะที่แอปพลิเคชันกำลังทำงานผ่านโค้ดเพื่อการจัดการ



รูปที่ 4.2 โค้ดเพื่อการจัดการทางฝั่งเซิร์ฟเวอร์